

ECE 222 Final Project Report
Design a Headphone Amplifier and Build a Prototype
Quang Nguyen and Sufi Hossain

I. Design

We started off with a dual stage amplifier design, after reading the literature and evaluating the parameters. We decided based on the components list and availability, we used OP1656 and BUF634A, one as the voltage gain stage and the other as the current buffer stage. The schematic (figure 1) of the two stage amplifier is only used for each channel (left and right) and the OPA1656 chip came with dual channel op amp configuration, and the schematic used is for a general one channel configuration. Our implementation consisted of two BUF634A chips and dual channel config OPA1656.

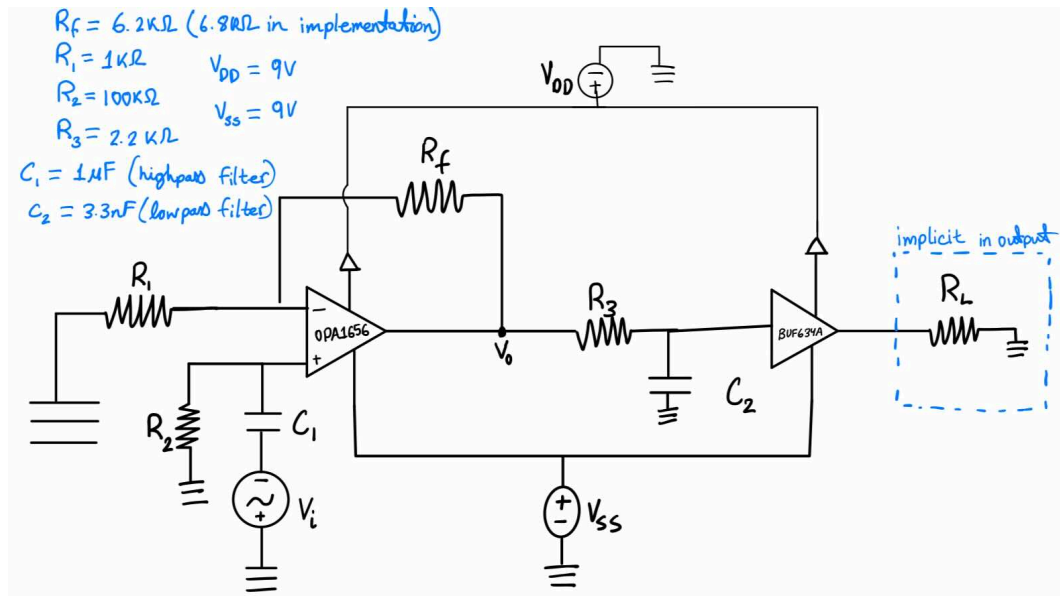


Figure 1

The values calculated (in blue) for the amplifier were calculated during the proposal phase of the project. The reasoning behind the implicit R_L is that it was stored behind the actual headphone, it was unnecessary to implement in the amplifier design (although during the designing phase we optimized it for the nominal value of 32k Ω). And the V_{SS} and V_{DD} were in an inverted grounded position to make sure they both shared the same ground and also were two distinct different power sources for the op amps and the buffers to power them in the proper V_+ and V_- channels. Our dual-channel design included 2x of 6.6k Ω , 1k Ω , 100k Ω , 2.2k Ω , 1 μF , and 3.3nF.

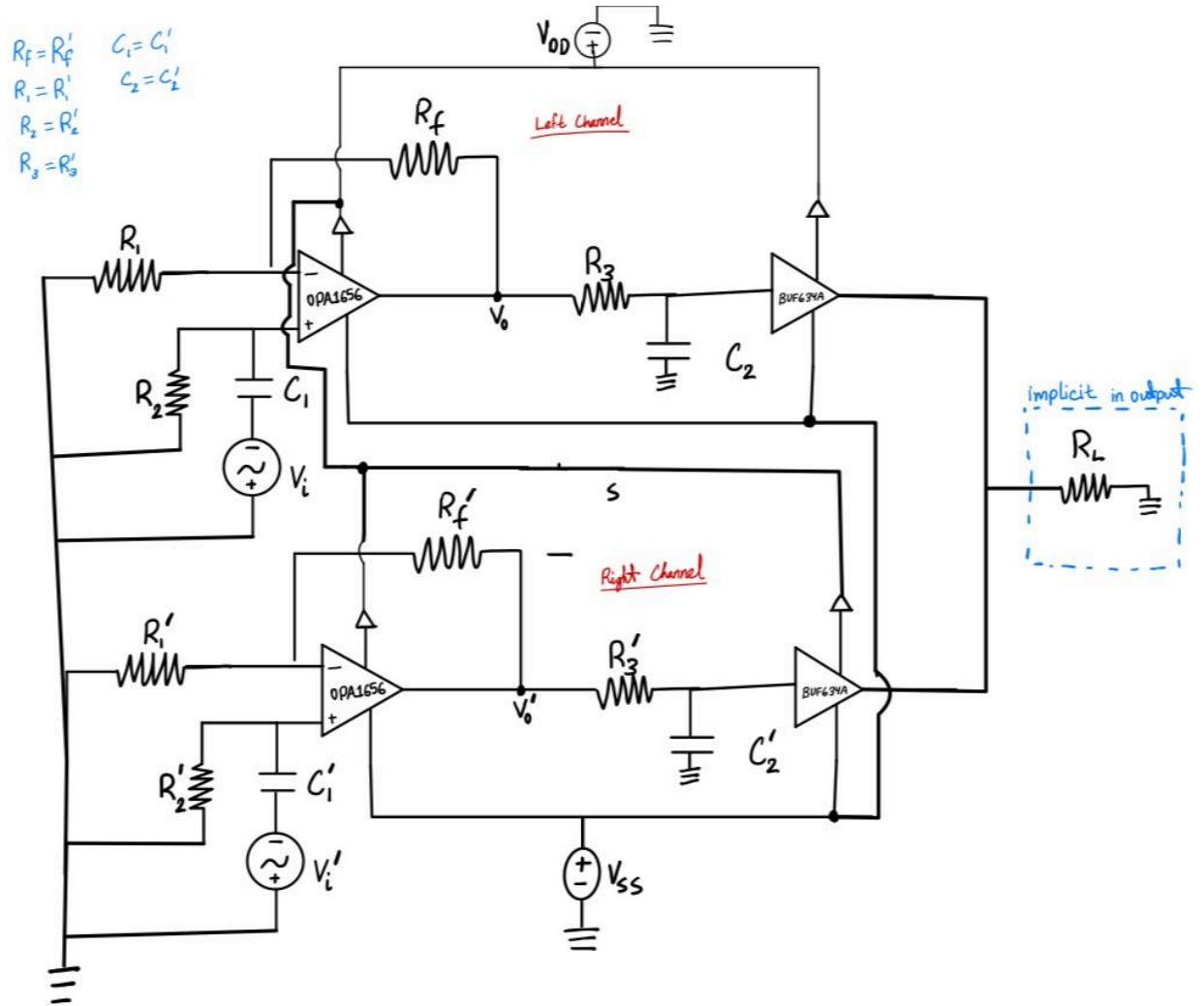


Figure 2: Schematic of dual-channel implementation

II. Simulation

We decided to run through a bunch of different simulations with our design and tests, and they seemed to have met the requirements of the lab with the configurations shown previously. Here are our HSPICE test_deck simulation results along with our test deck code.

Code for DUT1:

```
*** Description: Netlist for single channel audio amplifier ***
* DUT ports: in, out (input/output)
* vdd, vss (dual-rail power supplies)
.SUBCKT DUT in out vdd vss

* OPA1656 Non-Inverting Amplifier
X1 n_in n_neg vdd vss n_out OPA1656
* Input Network (High-Pass)
Cin in n_in 1uF
Rin n_in 0 100k
* Feedback Network
Rf n_out n_neg 6.2k
R1 n_neg 0 1k
* Output Network
R2 n_out out 2.2k
C2 out 0 3nF

.include OPA1656.hsp
.ENDS
```

Code for DUT:

```
*** Description: Netlist for 2 stage audio amplifier ***
.SUBCKT DUT in out vdd vss

* OPA1656 Non-Inverting Amplifier
X1 n_in n_neg vdd vss n_out OPA1656
X2 p_in p_out vdd vss BUF634A
* Input Network (High-Pass)
Cin in n_in 1uF
Rin n_in 0 100k
* Feedback Network
Rf n_out n_neg 6.2k
R1 n_neg 0 1k
* Output Network
R2 n_out out 2.2k
C2 out 0 3.3nF
```

```
*R3 p_out out 32 *(Implicit)
```

```
.include OPA1656.hsp  
.include buf634a_a.hsp  
.ENDS
```

Code for TEST deck4, named DUT2.hsp:

```
*** Description: Netlist for single channel audio amplifier ***
```

```
***FOR TEST DECK 4****
```

```
*vdd, vss (dual-rail power supplies)
```

```
.SUBCKT DUT in out vdd vss
```

```
* Input Network (High-Pass with proper biasing)
```

```
Cc1 in vi+ 1uF
```

```
Rin vi+ 0 100k
```

```
E12 vi2 0 VCVS vo1 0 1
```

```
* OPA1656 Non-Inverting Amplifier
```

```
X1 vi+ n_neg vdd vss n_out OPA1656
```

```
X2 vi2 out vdd vss BUF634A
```

```
* Feedback Network
```

```
Rf n_out n_neg 6.2k
```

```
R1 n_neg 0 1k
```

```
* Output Network
```

```
R2 n_out vo1 2.2k
```

```
C2 vo1 0 3nF
```

```
.include OPA1656.hsp
```

```
.include buf634a_a.hsp
```

```
.ENDS
```

RESULTS FROM TEST DECKS:

Test Decks for Small-Signal Gain and 3dB Bandwidth:

```
1a_AC_TF.ma0  
1 $DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0  
2 .TITLE '**** ece 222 course project ****'  
3 gain_mid gain_db bw_f1 bw_f2  
4 bw_3db temper alter#  
5 3.5959 11.1161 1.5955 1.059e+04  
6 1.059e+04 27.0000 1  
7
```

a)

Figure 3

b)

1	\$DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0			
2	.TITLE '**** ece 222 course project ****'			
3	gain_mid	gain_db	bw_f1	bw_f2
4	bw_3db	temper	alter#	
5	6.4127	16.1408	1.5946	2.398e+04
6	2.397e+04	27.0000	1	
7				

Figure 4

Test Deck for Small-Signal Input Impedance:

1	\$DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0			
2	.TITLE '**** ece 222 course project ****'			
3	zin_1k	zin_5k	zin_10k	zin_15k
4	zin_20k	temper	alter#	
5	1.000e+05	1.000e+05	1.000e+05	1.000e+05
6	1.000e+05	27.0000	1	
7				

Figure 5

Test Deck for Small Signal Output Impedance:

1	\$DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0			
2	.TITLE '**** ece 222 course project ****'			
3	zout_1k	zout_5k	zout_10k	zout_15k
4	zout_20k	temper	alter#	
5	3.9288	3.9355	3.9437	3.9437
6	3.9480	27.0000	1	
7				

Figure 6

Test Deck for Output Power and Power Consumption:

1	\$DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0			
2	.TITLE '**** ece 222 course project ****'			
3	thd_db	sndr_db	temper	alter#
4	-101.7705	4.3613	27.0000	1
5				

Figure 7

Test Deck for THD Analysis:

```
5_Transient_THD.mt0
1  $DATA1 SOURCE='PrimeSim HSPICE' VERSION='T-2022.06-SP1-2 win64' PARAM_COUNT=0
2  .TITLE '**** ece 222 course project ****'
3  thd_db      sndr_db      temper      alter#
4  -101.7705    4.3613      27.0000     1
5
```

Figure 8

Test Deck for Audio Response:

Note: The audio conversion and mixing was done in python with TA Sasanka's help and permission, due to a permission settings on the laptop, however, the audio conversion into a waveform was done in MATLAB, again with help from Sasanka. The audio file of the final conversion was uploaded as a part of the assignment in blackboard.

Audio Conversion Waveform:

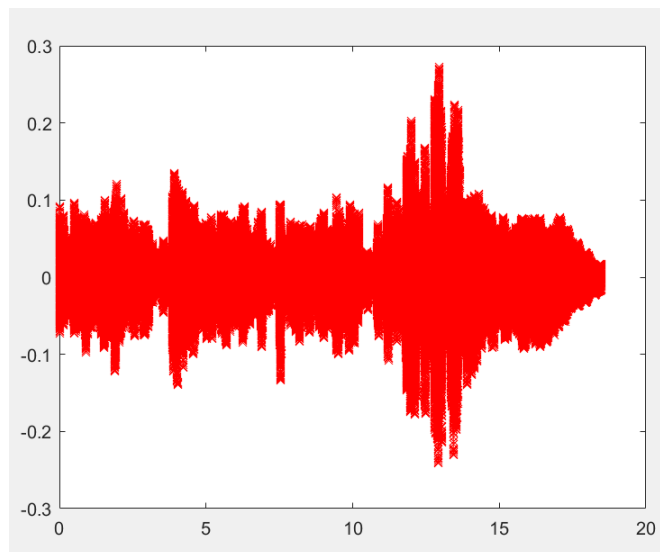


Figure 9

CODE:

SOUND CONVERSION (Python):

```
import re
import wave
import struct
import numpy as np

# --- Configuration ---
LEFT_PWL = 'recording_left.pwl0_tr0'
RIGHT_PWL = 'recording_right.pwl1_tr0'
OUTPUT_FILE = 'stereo_output.wav'
FS = 44100 # Sampling rate in Hz
```

```

# Multipliers for HSPICE unit suffixes
default_multipliers = {
    't': 1e12, 'g': 1e9, 'meg': 1e6, 'k': 1e3,
    'm': 1e-3, 'u': 1e-6, 'n': 1e-9, 'p': 1e-12,
    'f': 1e-15, 'a': 1e-18
}

def parse_value(token):
    m =
    re.fullmatch(r"([+-]?(?:\d+\.\d*|\.\d+)(?:[eE][+-]?\d+)?)([A-Za-z]+)?",
    token)
    if not m:
        raise ValueError(f"Cannot parse token '{token}'")
    num_str, suf = m.groups()
    val = float(num_str)
    if suf:
        s = suf.lower()
        if s not in default_multipliers:
            raise ValueError(f"Unknown suffix '{s}' in token '{token}'")
        val *= default_multipliers[s]
    return val

def read_pwl(filename):
    times, volts = [], []
    with open(filename, 'r') as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith('*'):
                continue
            m = re.match(r"^\\+?\\s*([+-]?[0-9].*?)\\s+([+-]?[0-9].*?)\\s*$", line)
            if not m:
                continue
            t_tok, v_tok = m.groups()
            times.append(parse_value(t_tok))
            volts.append(parse_value(v_tok))
    return np.array(times), np.array(volts)

def pwl_to_pcm(times, volts, fs, t_start, t_end):
    n_samples = int(np.ceil((t_end - t_start) * fs))
    t_uniform = np.linspace(t_start, t_end, n_samples, endpoint=False)
    v_uniform = np.interp(t_uniform, times, volts)
    max_val = np.max(np.abs(v_uniform))
    norm = v_uniform / max_val if max_val else v_uniform
    return np.int16(norm * 32767)

def main():
    # Load PWL data for both channels
    tL, vL = read_pwl(LEFT_PWL)
    tR, vR = read_pwl(RIGHT_PWL)

```

```

# Determine common start and end
t_start = min(tL[0], tR[0])
t_end = max(tL[-1], tR[-1])

# Generate PCM for each channel
pcm_L = pwl_to_pcm(tL, vL, FS, t_start, t_end)
pcm_R = pwl_to_pcm(tR, vR, FS, t_start, t_end)

# Interleave for stereo
n = len(pcm_L)
interleaved = np.empty(2 * n, dtype=np.int16)
interleaved[0::2] = pcm_L
interleaved[1::2] = pcm_R

# Write stereo WAV
with wave.open(OUTPUT_FILE, 'w') as wf:
    wf.setnchannels(2)
    wf.setsampwidth(2)
    wf.setframerate(FS)
    wf.writeframes(struct.pack('<' + 'h'*(2*n), *interleaved))

print(f"Wrote stereo WAV '{OUTPUT_FILE}' ({n} samples per channel at {FS}
Hz)")

if __name__ == '__main__':
    main()

```

Waveform Output (MATLAB):

```

leftFile = 'recording_left.pwl0_tr0';
rightFile = 'recording_right.pwl1_tr0';
function [timeVec, voltVec] = readPWL(filename)
    fid = fopen(filename, 'rt');
    if fid < 0
        error('Cannot open file: %s', filename);
    end
    tline = fgetl(fid);
    while ischar(tline)
        str = strtrim(tline);
        if isempty(str)
            tline = fgetl(fid);
            continue;
        end
        if ~isempty(regexpi(str, '[\+|-\.0-9]', 'once'))
            % First data line found
            break;
        end
        tline = fgetl(fid);
    end

    data = [];
    while ischar(tline)
        str = strtrim(tline);
        if isempty(str)
            tline = fgetl(fid);
        end
    end

```



```

        continue;
    end
    parts = strsplit(str);
    if numel(parts) >= 2
        data = [data; parts(1), parts(2)]; %#ok<AGROW>
    end
    tline = fgetl(fid);
end
fclose(fid);

if isempty(data)
    timeVec = [];
    voltVec = [];
    return;
end

nRows = size(data,1);
timeVec = zeros(nRows,1);
voltVec = zeros(nRows,1);
for i = 1:nRows
    token = data{i,1};
    suffix = '';
    if isletter(token(end))
        suffix = lower(token(end));
        numStr = token(1:end-1);
    else
        numStr = token;
    end
    val = str2double(numStr);
    switch suffix
        case 'p'
            val = val * 1e-12;
        case 'n'
            val = val * 1e-9;
        case 'u'
            val = val * 1e-6;
        case 'm'
            val = val * 1e-3;
        case ''
            otherwise
                error('Unknown time suffix "%s" in file %s', suffix, filename);
    end
    timeVec(i) = val;

    token = data{i,2};
    suffix = '';
    if isletter(token(end))
        suffix = lower(token(end));
        numStr = token(1:end-1);
    else
        numStr = token;
    end
    val = str2double(numStr);
    switch suffix
        case 'p'

```

```

        val = val * 1e-12;
    case 'n'
        val = val * 1e-9;
    case 'u'
        val = val * 1e-6;
    case 'm'
        val = val * 1e-3;
    case ''
    otherwise
        error('Unknown voltage suffix "%s" in file %s', suffix,
filename);
    end
    voltVec(i) = val;
end
end
[leftTime, leftVolt] = readPWL(leftFile);
[rightTime, rightVolt] = readPWL(rightFile);
[tL, idxL] = unique(leftTime);
vL = leftVolt(idxL);
[tR, idxR] = unique(rightTime);
vR = rightVolt(idxR);
Fs = 44100;
t_end = max(tL(end), tR(end));
tNew = (0 : 1/Fs : t_end).';
leftRes = interp1(tL, vL, tNew, 'pchip');
rightRes = interp1(tR, vR, tNew, 'pchip');
if max(abs(leftRes)) ~= 0
    leftRes = leftRes / max(abs(leftRes));
end
if max(abs(rightRes)) ~= 0
    rightRes = rightRes / max(abs(rightRes));
end
stereoAudio = [leftRes, rightRes];
audiowrite('amp_out_audio.wav', stereoAudio, Fs);
save('amp_out.mat', 'tNew', 'leftRes', 'rightRes', 'Fs');
figure;
subplot(2,1,1);
plot(leftTime, leftVolt, 'b.-'); hold on;
plot(tNew, leftRes, 'r');
xlabel('Time (s)'); ylabel('Left Channel Voltage');
legend('Raw Left', 'Resampled Left');
title('Left Channel: Raw vs Resampled');
subplot(2,1,2);
plot(rightTime, rightVolt, 'b.-'); hold on;
plot(tNew, rightRes, 'r');
xlabel('Time (s)'); ylabel('Right Channel Voltage');
legend('Raw Right', 'Resampled Right');
title('Right Channel: Raw vs Resampled');
% 11. Compare right channel to original Jan.wav if available
if exist('Jan.wav', 'file')
    [janData, FsJan] = audioread('Jan.wav');
    tJan = (0:length(janData)-1)/FsJan;
    figure;
    plot(tNew, rightRes, 'b');
    hold on;

```

```

if size(janData,2) >= 2
    plot(tJan, janData(:,2), 'g--');
    legend('Resampled PWL Right','Original Jan Right');
else
    legend('Resampled PWL Right');
end
xlabel('Time (s)'); ylabel('Amplitude');
title('Resampled PWL Right vs Original Jan.wav Right');
else
    warning('Jan.wav not found: skipping comparison plot.');
```

III. Building

After the simulation through HSPICE, and knowing what values we should aim for, we modeled the circuit on a breadboard (Figure 10), and tested it for all the values, and after verifying everything worked we used a perforated board and designed and soldered the entire final circuit (Figure 11). We had a list of components in addition to the ones listed previously: 2 x 3.5 mm female jack, 2 x 9v alkaline battery, and 2 x 9v battery clip. Finally we tested it again on the actual designed circuit, we manually did all the tests that were simulated on the test_decks through HSPICE and we received positive results, and it was working. It was demonstrated to the TAs and we received credit for it.

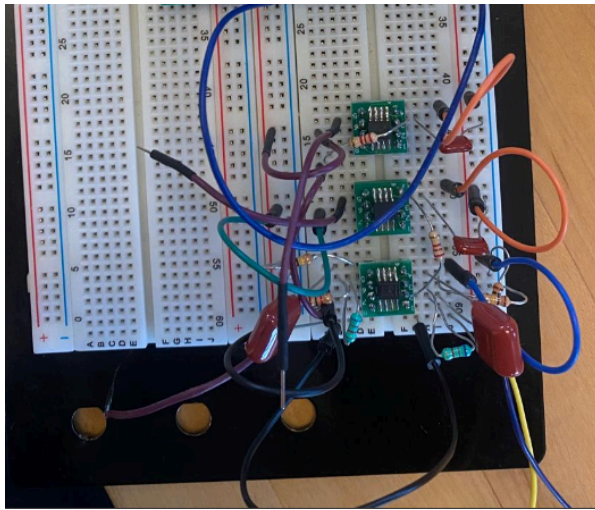


Figure 10

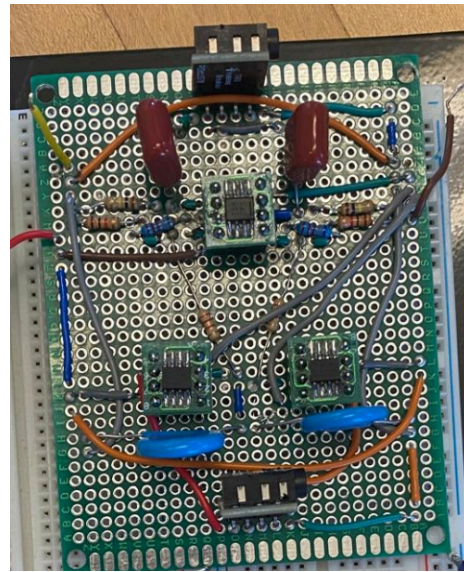


Figure 11

After testing the circuit we can see that the gain is working, and the signal is giving proper values (figure #), then we moved onto the testing portion. Using the source generator with 1V_{pp} amplitude and 1kHz frequency, both the gains from the left and right channel are around 8 V/V, which is correct with the intended design ($R_f = 6.8\text{k}\Omega$, $R_{in} = 1\text{k}\Omega$, which result to a gain = $1 + R_f/R_1 = 8 \text{ V/V}$)



Figure 12

Small-Signal Gain and 3dB Bandwidth for our circuit: (Within margin to simulated values)

Our 3dB measurements were about 21.5dB with cutoff frequency at 0.5Hz and 80Hz

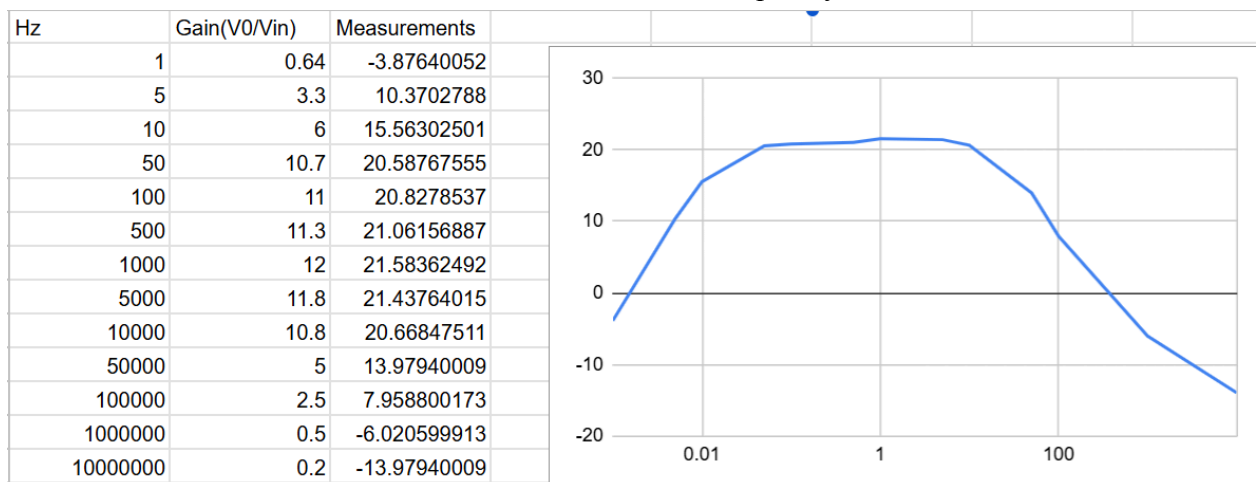


Figure 13

Small-Signal Input Impedance and Output Impedance: (Within margin to simulated values)

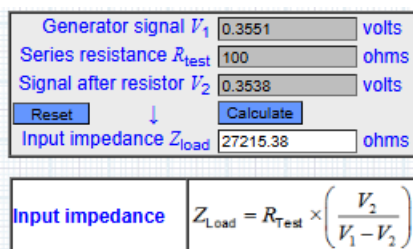


Figure 14: Small Signal Input Impedance

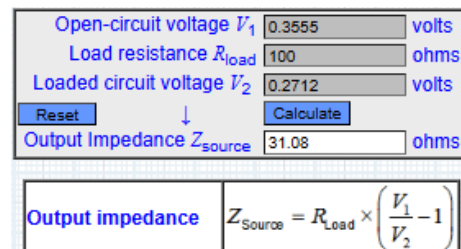


Figure 15: Small Signal Output Impedance

Output Power and Power Consumption:

$$\begin{aligned} R_L &= 32\Omega \\ V_{out, \text{peak to peak}} &= 8\text{ V} \\ V_{out, \text{peak}} &= \frac{8}{2} = 4\text{ V} \\ V_{out, \text{RMS}} &= \frac{V_{\text{peak}}}{\sqrt{2}} = 2.82\text{ V} \end{aligned} \quad \left. \begin{array}{l} \rightarrow P_{out} = \frac{V_{\text{RMS}}^2}{R_L} = 0.25\text{ W} \\ \rightarrow \text{meets requirement of } 100\text{ mW} \end{array} \right\}$$

$$\begin{aligned} P_{\text{consumption}} &= |V_{DD} \times I_{DD}| + |V_{SS} \times I_{SS}| \\ &\rightarrow \text{Measured current from OPA1656} \approx 2.6\text{ mA} \\ &\rightarrow \text{Measured current passing through BUF634A} \approx 10\text{ mA} \\ I_{\text{total}} &\approx 2(2.6\text{ mA}) + 2(10\text{ mA}) = 25.2\text{ mA} \\ I_{\text{load}} &= \frac{V_{\text{RMS}}}{R_L} \approx 88.4\text{ mA} \\ I_{DD} = I_{SS} &= I_{\text{total}} + I_{\text{load}} = 113.6\text{ mA} \\ P_{\text{consumption}} &= 2.045\text{ W} \end{aligned}$$

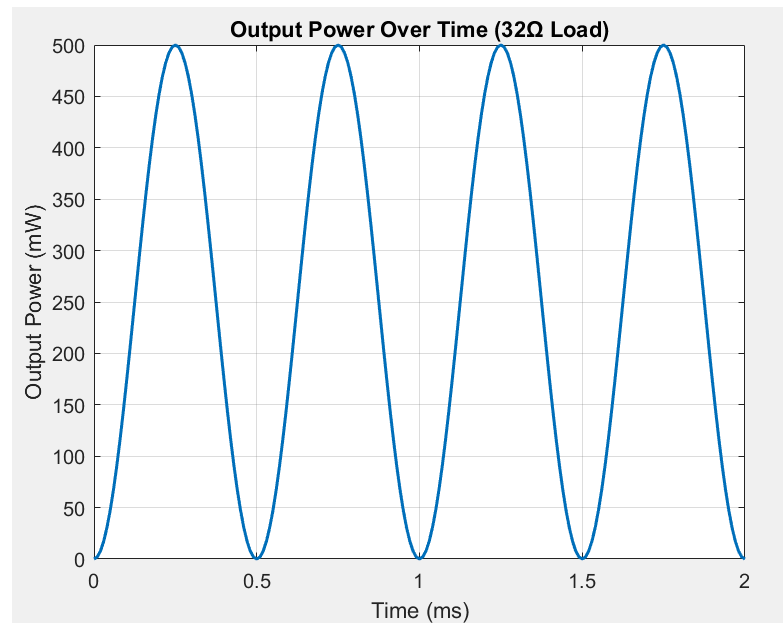


Figure 16: Simulated output power vs. time

THD Analysis:

Fundamental (1kHz): 2.75 V

2nd Harmonic (2kHz): 17.5 mV

3rd Harmonic (3kHz): 8.0 mV

4th Harmonic (4kHz): 5.0 mV

Calculated THD: 0.72%

THD (dB): -42.82 dB

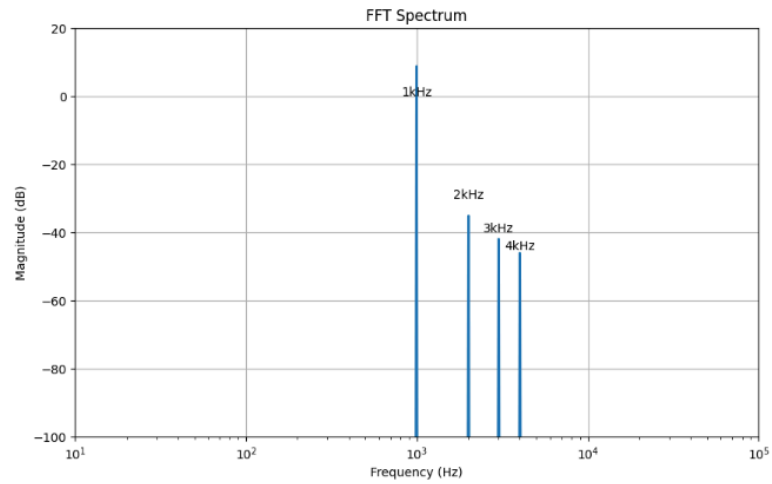


Figure 17

Test Deck for Audio Response:

It was shown to the TA with a headphone in fully working condition and played the music that was in the test deck file and another from youtube which is shown below (figure 18), and the TA tested it.

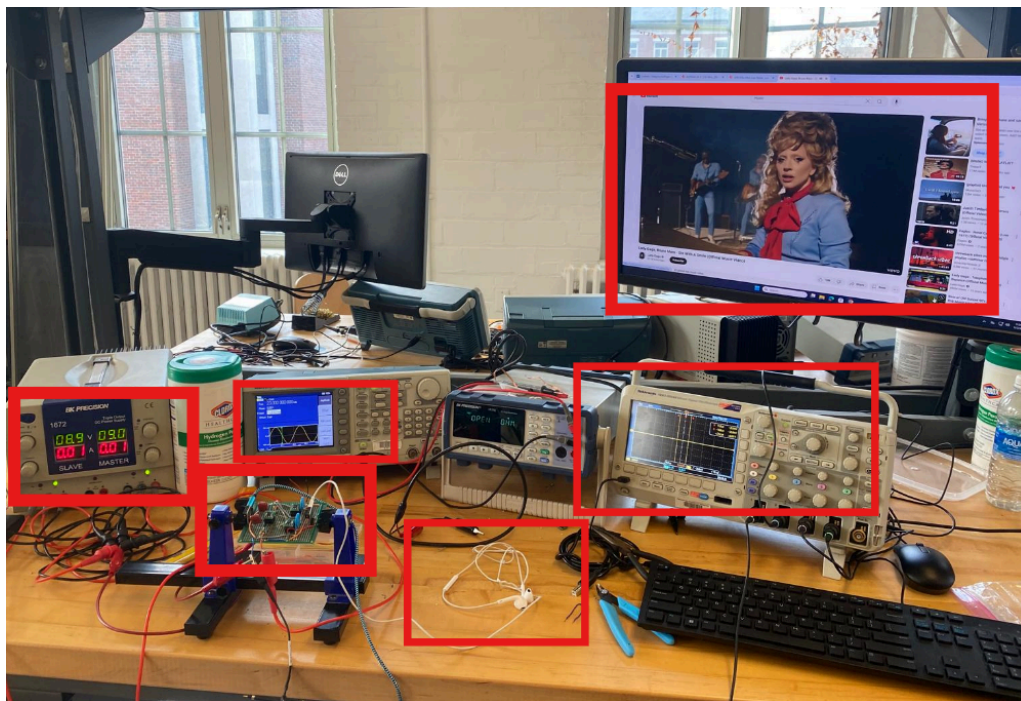


Figure 18

IV. Conclusion

In this project, we successfully designed, simulated, built, and tested a two-stage headphone amplifier using the OPA1656 and BUF634A components. Through careful design of the voltage gain and buffer stages, we achieved an output power exceeding 100 mW into a nominal 32 Ω load, while maintaining a wide bandwidth suitable for high-fidelity audio applications. Simulation and measurement results confirmed that the amplifier met key specifications, including input and output impedance targets, gain accuracy, and low total harmonic distortion (THD). The final FFT analysis showed a clean spectral output with a THD of approximately 0.72%, demonstrating good linearity and performance. Furthermore, the amplifier was successfully built and demonstrated on a breadboard and a soldered protoboard. Overall, this project provided valuable hands-on experience in analog circuit design, simulation, hardware prototyping, and performance verification.